

Relational Plot: FacetGrid

Relational plots are used for visualizing the statistical relationship between the data points. Visualization is necessary because it allows the human to see trends and patterns in the data. The process of understanding how the variables in the dataset relate each other and their relationships are termed as Statistical analysis.

FacetGrid class helps in visualizing distribution of one variable as well as the relationship between multiple variables separately within subsets of your dataset using multiple panels. A FacetGrid can be drawn with up to three dimensions ? row, col, and hue. The first two have obvious correspondence with the result in array of axes; think of the hue variable as a third dimension along a depth axis, where different levels are plotted with different colors. FacetGrid object takes a dataframe as input and the names of the variables that will form the row, column, or hue dimensions of the grid. The variables should be categorical and the data at each level of the variable will be used for a facet along that axis.

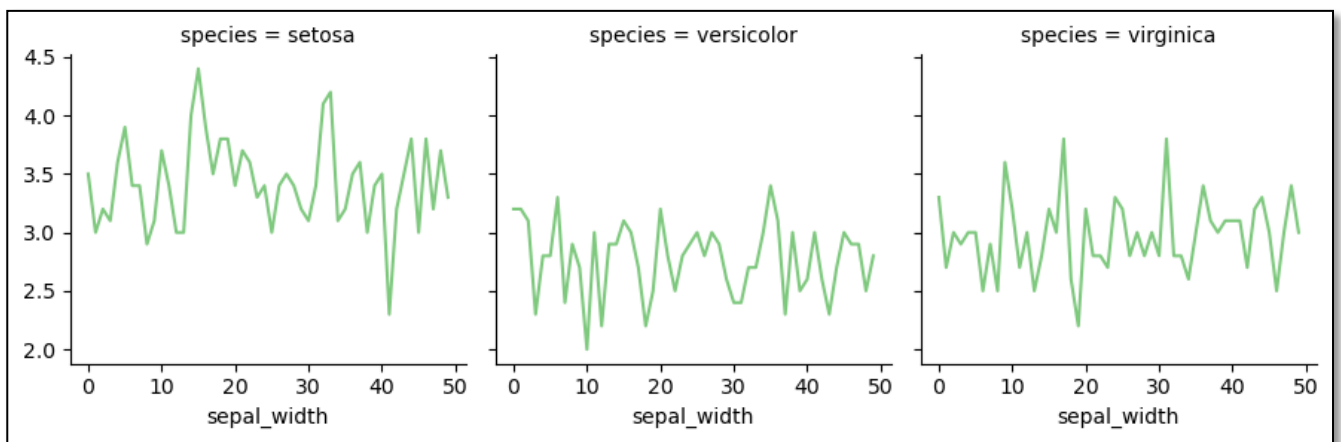
Syntax:

```
seaborn.FacetGrid( data, \*\*kwargs)
```

```
# importing packages
import seaborn as sns
import matplotlib.pyplot as plt

# loading dataset
data = sns.load_dataset("iris")

plot1 = sns.FacetGrid(data, col="species")
plot1.map(plt.plot, "sepal_width")
plt.show()
```



Creating Different Plot Types in Seaborn

Scatter Plot

The scatter plot is a mainstay of statistical visualization. It depicts the joint distribution of two variables using a cloud of points, where each point represents an observation in the dataset. This depiction allows the eye to infer a substantial amount of information about whether there is any meaningful relationship between them. It is plotted using the `scatterplot()` method.

Syntax:

```
seaborn.scatterplot(x=None, y=None, data=None, **kwargs)
```

```
# importing packages
```

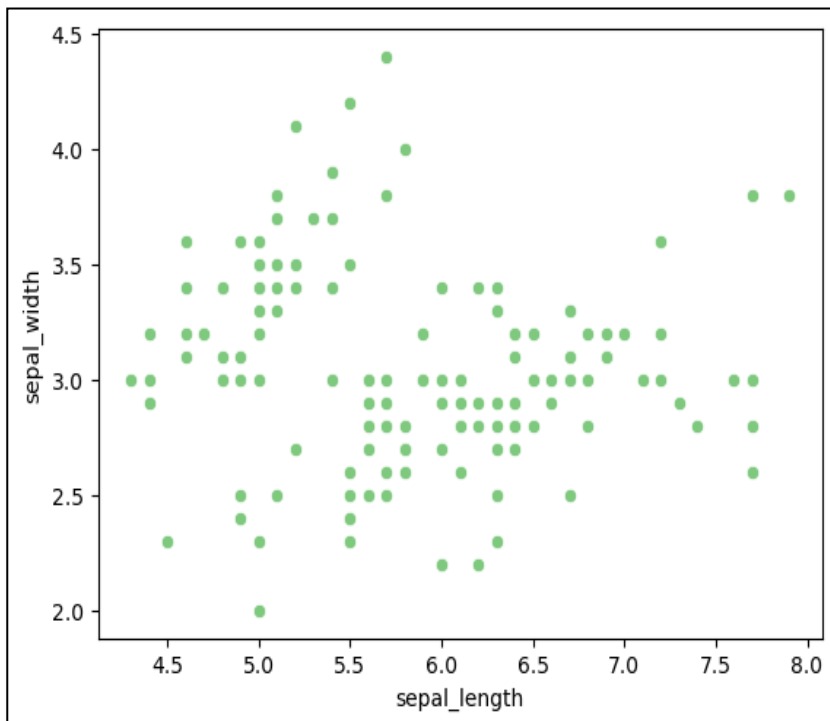
```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# loading dataset
```

```
data = sns.load_dataset("iris")
```

```
sns.scatterplot(x='sepal_length',y='sepal_width', data=data)  
plt.show()
```



Line Plot

For certain datasets, you may want to consider changes as a function of time in one variable, or as a similarly continuous variable. In this case, drawing a line-plot is a better option. It is plotted using the `lineplot()` method.

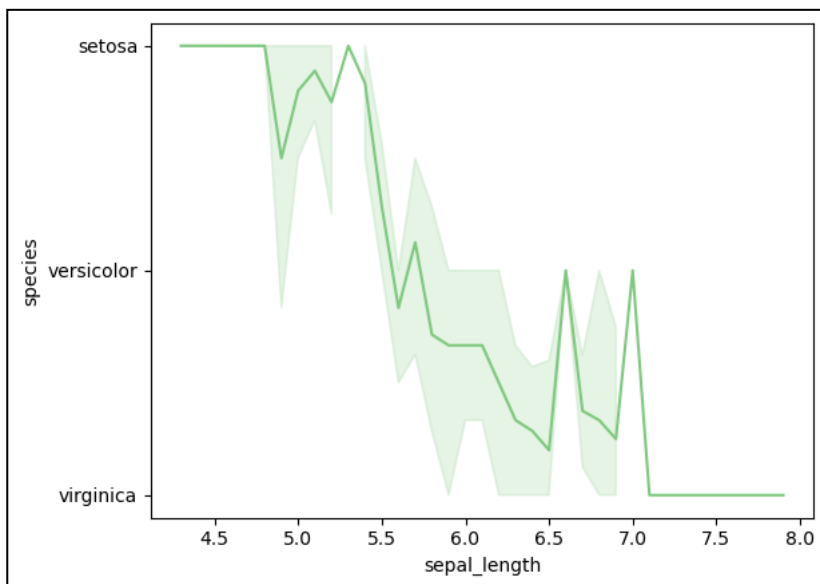
Syntax:

```
seaborn.Lineplot(x=None, y=None, data=None, **kwargs)
```

```
# importing packages
```

```
import seaborn as sns
import matplotlib.pyplot as plt
# loading dataset
dataset = sns.load_dataset("iris")

sns.lineplot(x='sepal_length', y='species', data=dataset)
plt.show()
```



Categorical Plots

Categorical Plots are used where we have to visualize relationship between two numerical values. A more specialized approach can be used if one of the main variable is categorical which means such variables that take on a fixed and limited number of possible values.

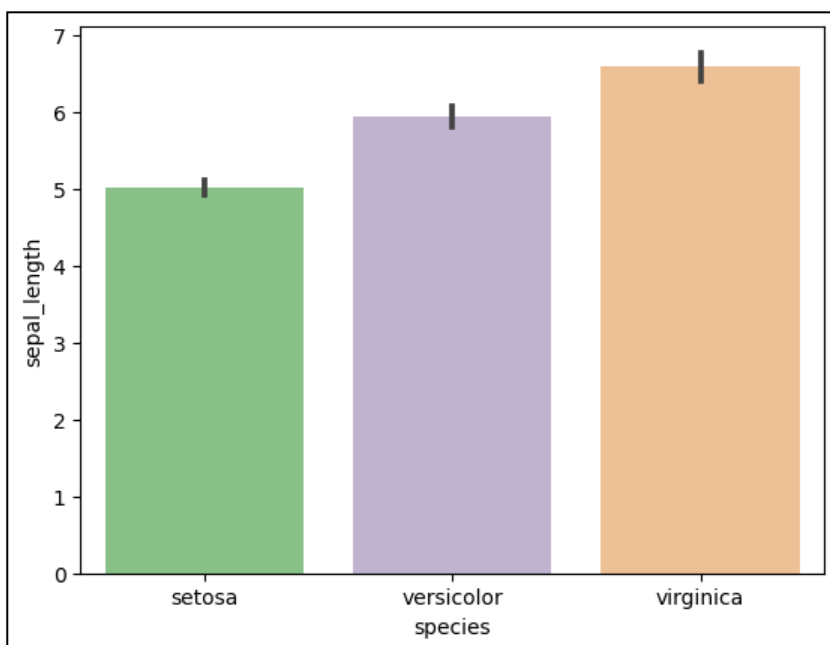
Bar Plot

A barplot is basically used to aggregate the categorical data according to some methods and by default it's the mean. It can also be understood

as a visualization of the group by action. To use this plot we choose a categorical column for the x axis and a numerical column for the y axis and we see that it creates a plot taking a mean per categorical column. It can be created using the `barplot()` method.

Syntax:

```
barplot([x, y, hue, data, order, hue_order, ...])  
  
# importing packages  
import seaborn as sns  
import matplotlib.pyplot as plt  
  
# loading dataset  
dataset = sns.load_dataset("iris")  
sns.barplot(x='species', y='sepal_length', data=dataset)  
plt.show()
```



Catplot

`catplot()` is a **categorical plot function** in Seaborn that lets you visualize relationships between a **categorical variable** and other variables. It supports

many **types of plots** (called "kinds"), and allows grouping, coloring, and even faceting (splitting into multiple subplots).

Syntax:

```
sns.catplot(x,y,data,hue,kind.....)
```

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
dataset= sns.load_dataset("penguins")
print(dataset.head(2))
```

Output:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	\
0	Adelie	Torgersen	39.1	18.7	181.0	
1	Adelie	Torgersen	39.5	17.4	186.0	

	body_mass_g	sex
0	3750.0	Male
1	3800.0	Female

```
# Draw a nested barplot by species and sex
```

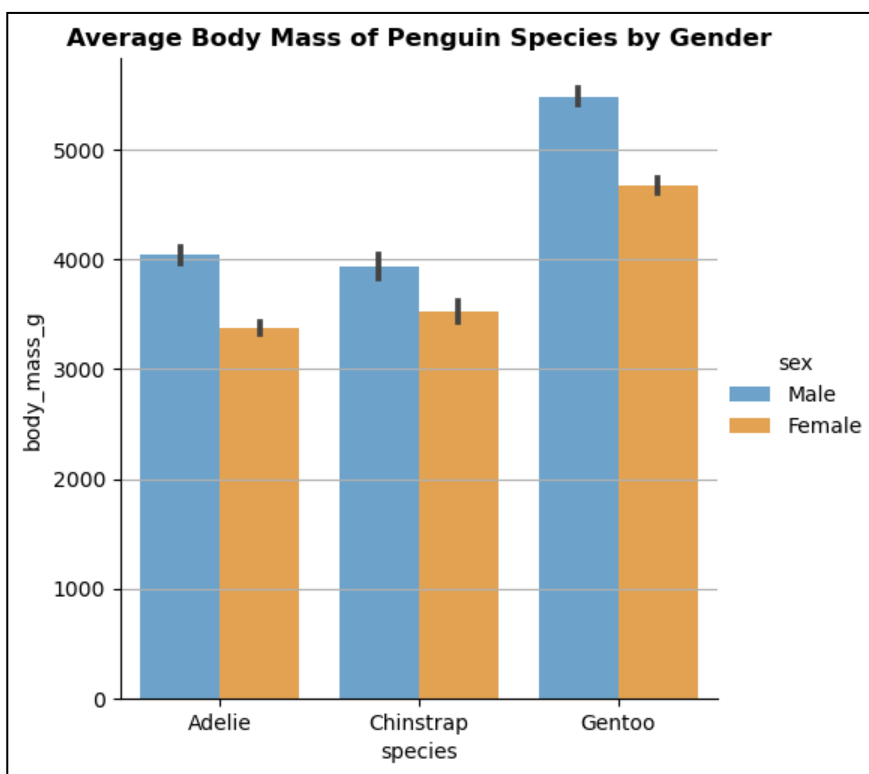
```
sns.catplot(
    data=dataset,
    kind="bar",
    x="species",
    y="body_mass_g",
    hue="sex",
```

```
palette={"Male": "#5DA5DA", "Female": "#FAA43A"})
```

```
plt.title("Average Body Mass of Penguin Species by  
Gender", fontweight='bold')
```

```
plt.grid(axis='y')
```

```
plt.show()
```



Count Plot

A countplot basically counts the categories and returns a count of their occurrences. It is one of the most simple plots provided by the seaborn library. It can be created using the `countplot()` method.

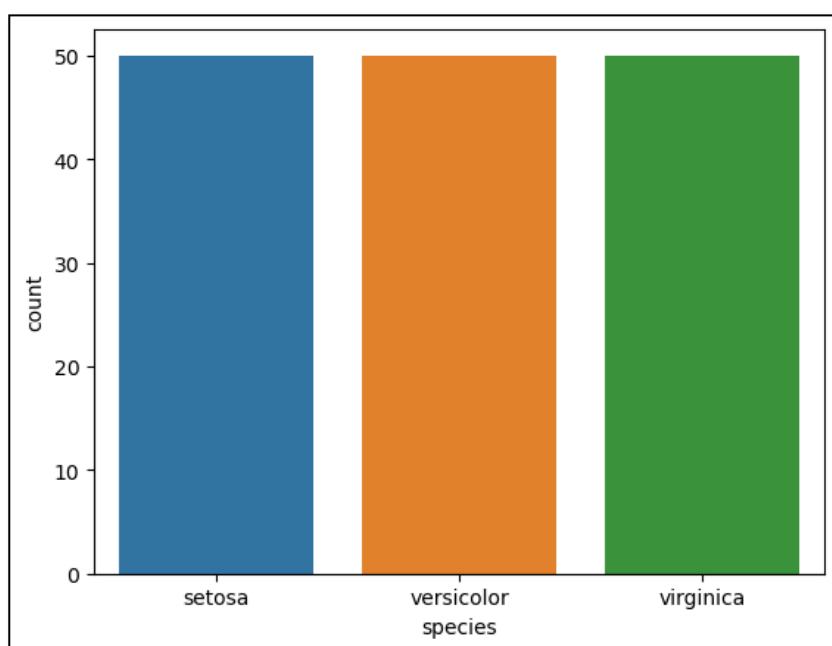
Syntax:

```
countplot([x, y, hue, data, order, ...])
```

```
# importing packages
import seaborn as sns
import matplotlib.pyplot as plt

# loading dataset
dataset = sns.load_dataset("iris")

sns.countplot(x='species', data=dataset)
plt.show()
```



Box Plot

A boxplot is sometimes known as the box and whisker plot. It shows the distribution of the quantitative data that represents the comparisons between variables. boxplot shows the quartiles of the dataset while the whiskers extend to show the rest of the distribution i.e. the dots indicating the presence of outliers. It is created using the `boxplot()` method.

Syntax:

```

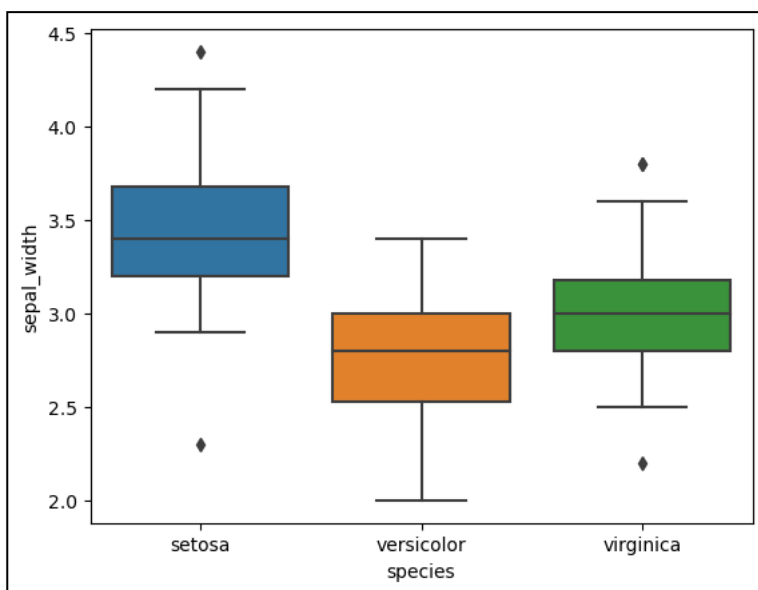
boxplot([x, y, hue, data, order, hue_order, ...])

# importing packages
import seaborn as sns
import matplotlib.pyplot as plt

# loading dataset
data = sns.load_dataset("iris")

sns.boxplot(x='species', y='sepal_width', data=data)
plt.show()

```



Violin Plot

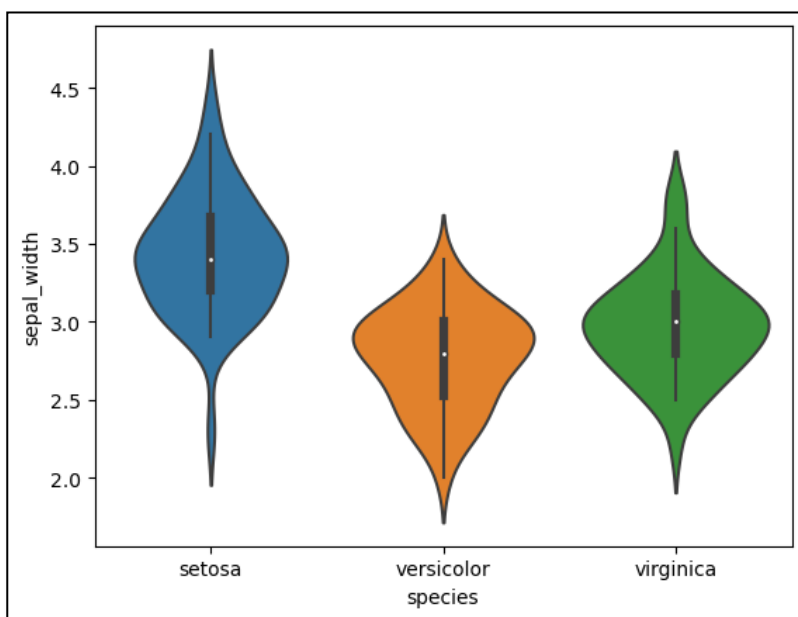
It is similar to the boxplot except that it provides a higher, more advanced visualization and uses the kernel density estimation to give a better description about the data distribution. It is created using the `violinplot()` method.

Syntax:

```
violinplot([x, y, hue, data, order, ...])
```

```
# importing packages
import seaborn as sns
import matplotlib.pyplot as plt

# loading dataset
dataset = sns.load_dataset("iris")
sns.violinplot(x='species', y='sepal_width', data=dataset)
plt.show()
```



Distribution Plots

Distribution Plots are used for examining univariate and bivariate distributions meaning such distributions that involve one variable or two discrete variables.

Refer to the below article to get detailed information about the distribution plots.

histplot

Distplot is used basically for univariate set of observations and visualizes it

through a histogram i.e. only one observation and hence we choose one particular column of the dataset. It is plotted using the `distplot()` method.

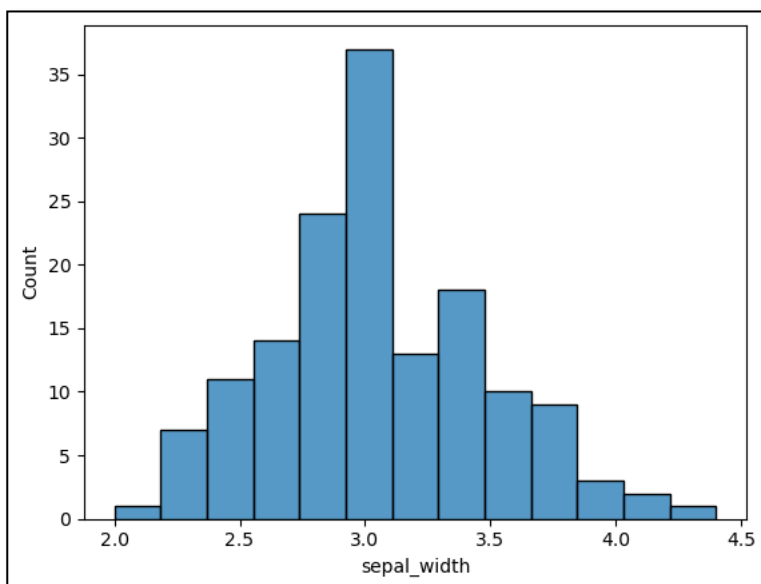
Syntax:

```
histplot(a[, bins, hist, kde, rug, fit, ...])
```

```
# importing packages
import seaborn as sns
import matplotlib.pyplot as plt

# loading dataset
dataset = sns.load_dataset("iris")

sns.histplot(dataset['sepal_width'])
plt.show()
```



Pairwise Relationships: `pairplot()`

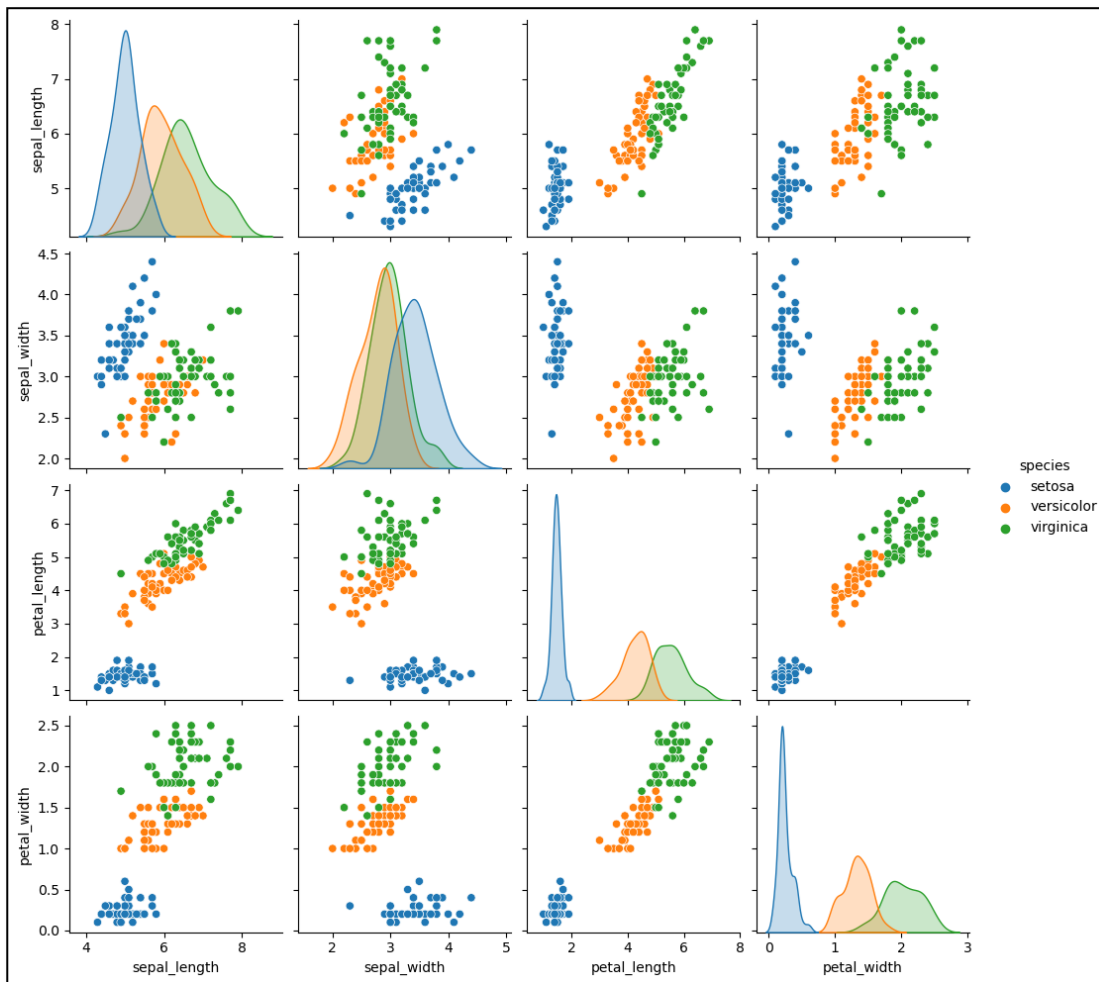
Pairplot represents pairwise relation across the entire dataframe and supports an additional argument called `hue` for categorical separation. What it does

basically is create a jointplot between every possible numerical column and takes a while if the dataframe is really huge. It is plotted using the pairplot() method.

Syntax:

```
pairplot(data[, hue, hue_order, palette, ...])
```

```
# importing packages
import seaborn as sns
import matplotlib.pyplot as plt
# loading dataset
dataset = sns.load_dataset("iris")
sns.pairplot(data=dataset, hue='species')
plt.show()
```



Matrix Plot

A matrix plot means plotting matrix data where color coded diagrams shows rows data, column data and values. It can shown using the heatmap and clustermat.

Heatmap for Correlation Matrix

Heatmap is defined as a graphical representation of data using colors to visualize the value of the matrix. In this, to represent more common values or higher activities brighter colors basically reddish colors are used and to represent less common or .activity values, darker colors are preferred. it can be plotted using the heatmap() function.

```
# import the necessary libraries
import seaborn as sns
import matplotlib.pyplot as plt

# Load the tips dataset
dataset = sns.load_dataset('tips')

# Select only numeric columns
numeric_data = dataset.select_dtypes(include=['float64',
'int64'])

# Calculate the correlation between the numeric columns
tc = numeric_data.corr()

# Create the heatmap for the correlation matrix
sns.heatmap(tc, annot=True, cmap='plasma', linecolor='black',
linewidths=1)

# Display the plot
plt.show()
```

